

**İSTANBUL TOPKAPI ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
PROJE RAPORU**

Adı: CEN306-Mobile Application Design and Development

Date:

**Students
or Team
Members
Who
Prepared
the
Project**

**Name-
Surname:
Signature:**

Ayşegül Baytekin

**Student
No:**

22040102036

**Name-
Surname:
Signature:**

Berke Portakal

**Student
No:**

22040102027

**Name-
Surname:
Signature:**

**Student
No:**

Programı:

Bilgisayar Mühendisliği (İng.)

(This section will be filled out by the Instructor.)

Subject of Evaluation

Evaluation Score

Flutter Mobile Application Project — Code Evaluation (70 Points)

Flutter Mobile Application Project — Project Report Evaluation (30 Points)

IMPORTANT: Maximum 10-15 pages, screenshots should occupy a maximum of one page in

total, screenshot images should be small, and no source code should be included.

CEN306-Mobile Application Design and Development

Final Exam Project Report

Project Title	Medician Reminder Mobile Application
Student Name-Surname	Ayşegül Baytekin – Berke Portakal
Student Number	22040102036 - 22040102027
Student Signature	
Course Name	CEN306 - Mobile Application Design and Development
Instructor	Dr. Yıldız Karadayı
Submission Date	[16/06/2026]
GitHub / Source Code Link	https://github.com/Bekoo10/medication_reminder_app.git
Demo Video YouTube Link	https://youtu.be/Nx0qB6b9nls?si=dz-zr0L_RRMNgDFS
Application Name	Medician Reminder

Submission reminder

Before submitting, replace all bracketed placeholders, insert your own screenshots, check captions and references, and make sure the report explains both the user-facing application and the technical architecture behind it.

Academic Integrity Statement

I/we confirm that this report and project were prepared by me/us and that all external sources, libraries, images, code snippets, and datasets are properly acknowledged.

How to Use This Template

Keep the section headings and delete guidance text if you prefer a cleaner final report, but make sure each expected topic is still covered.

Step	What to do
1	Replace every [placeholder] with your own project information.
2	Insert screenshots of the main screens and write explanations under each screenshot.
3	Explain your layered architecture and data-layer design, especially SQLite, DAO, and Repository usage.
4	Describe technical challenges, design decisions, and solutions applied during development.
5	Proofread the report for clarity, consistency, captions, and technical accuracy before submission.

Evaluation Alignment Checklist: 30 Points

The following checklist is aligned with the official Project Report Evaluation Criteria (30 points). Use it to verify that your report provides evidence for each scoring category.

Official report criterion	Points	Report section(s)	Evidence / self-check
Report Structure and Organization	5	Cover page; TOC; all main sections	Clear academic/technical organization; consistent headings, formatting, page numbers, captions, references, and no remaining placeholders.
Project Purpose and Scope	5	Sections 1-3	Purpose, problem addressed, target users, scope, core features, requirements, and use cases are clearly explained.
Explanation of Architecture and Technical Design	10	Sections 5-7	Layered architecture; UI/business/data separation; folder structure; model/entity objects; SQLite tables; DAO methods; Repository API; data flow; state flow; navigation/state management; technical decisions.
Screenshots and Screen Explanations	5	Section 4; Appendix A	Main functional screens with captions; each screen's purpose, displayed data, user actions, and navigation paths are explained.
Technical Reflection and Development Process	5	Sections 8-10	Challenges, causes, solutions, testing evidence, limitations, lessons learned, and future improvements are summarized.
Project Report Total	30	All sections	The report documents both the user-facing application and the technical implementation behind it.

Rubric reminder: Screenshots alone are not sufficient. Explain the technical structure, including data and state flow, model objects, SQLite table design, DAO functions, and Repository responsibilities. The submitted project should run successfully and should avoid unexplained ready-made templates or high-density AI-generated code.

Suggested Table of Contents

After completing the report, you may generate or update an automatic Table of Contents in Microsoft Word using References > Table of Contents.

1. Executive Summary	[page]
2. Project Introduction	[page]
3. Requirements and Use Cases.....	[page]
4. User Interface Design and Screens.....	[page]
5. Technical Architecture	[page]
6. Data Layer: SQLite, DAO, and Repository	[page]

7. Implementation Details.....	[page]
8. Testing and Quality Assurance.....	[page]
9. Challenges, Design Decisions, and Solutions	[page]
10. Conclusion and Future Work.....	[page]
11. References.....	[page]
12. Appendices.....	[page]

1. Executive Summary

Write a brief overview of your project. This section should help the reader understand the application without reading the full report.

Item	Short answer
Application name	Medician Reminder
Problem addressed	Missing medication times or making dosage errors due to the hustle of daily life, polypharmacy , or cognitive factors.
Target users	Individuals who need to manage their medication routines end-to-end through a digital mechanism.
Core technologies	Flutter, Dart, SQLite (sqflite), Provider (ChangeNotifier), Timer.
Main contribution	An application designed to minimize doses and automatically track medication inventory to increase overall treatment adherence.

2. Project Introduction

2.1 Project Purpose

The Medician Reminder mobile application is designed to allow individuals to manage their medication routines end-to-end through a digital mechanism. Its main goal is to minimize missed doses and automatically track medication inventory to increase overall treatment adherence.

2.2 Scope of the Project

The architectural scope of the application includes a dynamic and flexible scheduling module, a real-time timer and smart alarm mechanism, inventory and critical stock control, and patient compliance analytics.

2.3 Target Users and Usage Scenario

The target users are patients who require flexible calendar infrastructures for medications that need to be taken regularly every day, every other day, or on specific days of the week. A background periodic loop continuously checks the scheduled times. When a scheduled time arrives, an audio alert is triggered asynchronously, and the user can either snooze the alarm for 5 minutes or dismiss it completely. Upon marking a medication dose as "taken", the stock amount is automatically decreased by one unit.

2.4 Main Features

List the most important features. Use action-oriented descriptions, such as "Users can create, update, delete, and search notes".

Feature	Description	Related screen(s)
Dynamic and Flexible Scheduling	Allows users to define different periods (daily, alternate_days, specific_days) according to treatment protocol.	Add/Edit Medication Form
Real-Time Timer & Smart Alarm	Background loop checks times and triggers audio alerts (alarm.mp3) asynchronously.	Alarm / Dashboard
Inventory & Critical Stock Control	Decreases stock automatically when marked "taken" and checks critical stock thresholds.	Dashboard / Medication Detail
Feature	Description	Related screen(s)

Patient Compliance Analytics	Chronologically records taken or skipped actions to calculate treatment compliance rates.	History and Logs
------------------------------	---	------------------

3. Requirements and Use Cases

3.1 Functional Requirements

Functional requirements describe what the application must do. Keep each requirement measurable and testable.

ID	Functional requirement	Priority	Implemented?
FR-01	The user can define different periods according to each medication's treatment protocol.	High	Yes
FR-02	The system must trigger an audio alert asynchronously when the scheduled time arrives.	High	Yes
FR-03	The application must automatically decrease the current stock amount by one unit when a dose is marked as "taken".	High	Yes
FR-04	The system must chronologically record the user's retrospective actions (taken or skipped) in the dose_logs table.	High	Yes
FR-05	The application must calculate the user's treatment compliance rate between 0.0 and 1.0.	Medium	Yes

3.2 Non-Functional Requirements

Non-functional requirements describe qualities such as usability, performance, maintainability, and reliability.

Category	Requirement	How it is addressed
Usability	The user should have flexibility when a medication reminder occurs.	An audio alert (alarm.mp3) is triggered, and the user is provided with the flexibility to snooze the alarm for 5 minutes or dismiss it completely.
Performance	The UI must not lag or drop frames during database operations.	UI is isolated from data fetches; MedicationService handles asynchronous SQLite INSERT/UPDATE operations in the background and updates the UI via notifyListeners().
Reliability	The alarm system must not trigger duplicate alerts for the same medication dose.	A Set<String> _triggeredAlarms mechanism uses a unique composite key to ensure each medication alarm fires strictly once, preventing overlapping audio.
Category	Requirement	How it is addressed

Maintainability	Code should be organized into clearly separated layers to uphold the Single Responsibility Principle.	A strict layered architecture separates Presentation, Business/Domain, and Data layers. UI components are completely decoupled from direct database queries.
-----------------	---	--

3.3 Use Cases

Use case	Actor	Precondition	Main flow	Expected result
UC-01: Add new medication	User	The app is open on the Add/Edit Medication Form.	The user enters details (name, dosage, frequency, specific days, low stock threshold) and submits the form.	MedicationService.addMedication serializes the data and performs an asynchronous INSERT into the SQLite medications table via MedicationDao.
UC-02: Mark dose as taken	User	A medication is scheduled and visible to the user.	The user marks a medication dose as "taken".	A log record is inserted via DoseLogDao, inventory stock is decremented via MedicationDao, and in-memory lists reload to update the UI.
UC-03: Respond to alarm	User	A scheduled medication time arrives and the background timer triggers.	An audio alert (alarm.mp3) plays. The user chooses to snooze the alarm for 5 minutes or dismiss it completely.	The alarm is temporarily paused (snoozed) or stopped, and duplicate alarms are prevented via the _triggeredAlarms set.

4. User Interface Design and Screens

4.1 UI Design Approach

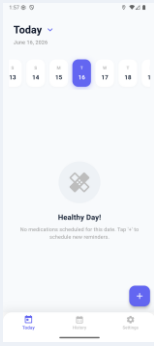
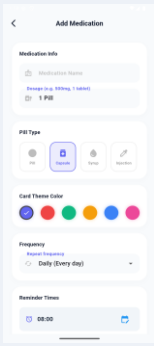
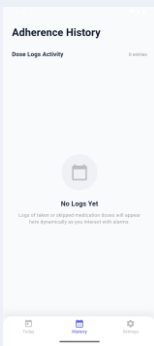
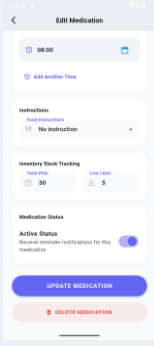
The interface visually utilizes the colorHex property stored in the database to color-coordinate the UI elements dynamically, as seen on the Medication Detail screen. The UI components are completely decoupled from direct database queries, ensuring a smooth user experience by communicating exclusively with the Business layer.

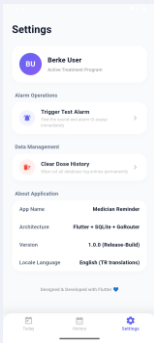
4.2 Navigation Flow

The primary landing page is the Dashboard. From the Dashboard, users can navigate to the Add/Edit Medication Form, History and Logs, or Settings. When a user taps on a specific medication from the list, navigation routes them to the Medication Detail screen, passing the Medication model object as an argument.

4.3 Main Screenshots and Explanations

Insert screenshots of your main screens. Each screenshot should have a caption and a clear explanation of its function.

Screen name	Screenshot placeholder	Purpose / main function	Navigation / user actions
Dashboard / Main Screen		This is the primary landing page. It displays the daily medication schedule and the user's overall adherence progress.	Fetches active medications for the current date. Rebuilds the interface synchronously without lagging when active alarms or stock reductions change.
Add/Edit Medication Form		Provides a data entry form for users to register a new medication protocol or edit an existing one.	Captures details (name, dosage, frequency, specific days, low stock threshold). Form submission triggers an asynchronous SQLite INSERT operation.
History and Logs		Acts as an audit trail for the user's actions, displaying a chronological list of all taken and skipped medications	The UI retrieves the data asynchronously by invoking methods that execute SQL queries ordered by the logged DateTime.
Medication Detail		Displays in-depth details such as instructions (e.g., "With water"), pillType, and the current stockCount.	Reached by tapping a specific medication from the list. Receives the Medication model object as a route argument.

Settings		Handles global application configurations and database management.	Includes administrative functions such as wiping the log history. Tapping the clear data option truncates the table and updates the global state.
----------	---	--	---

5. Technical Architecture

5.1 Overall Layered Architecture

Explain the layered architecture used in your Flutter application. You may replace the diagram below with your own architecture diagram.

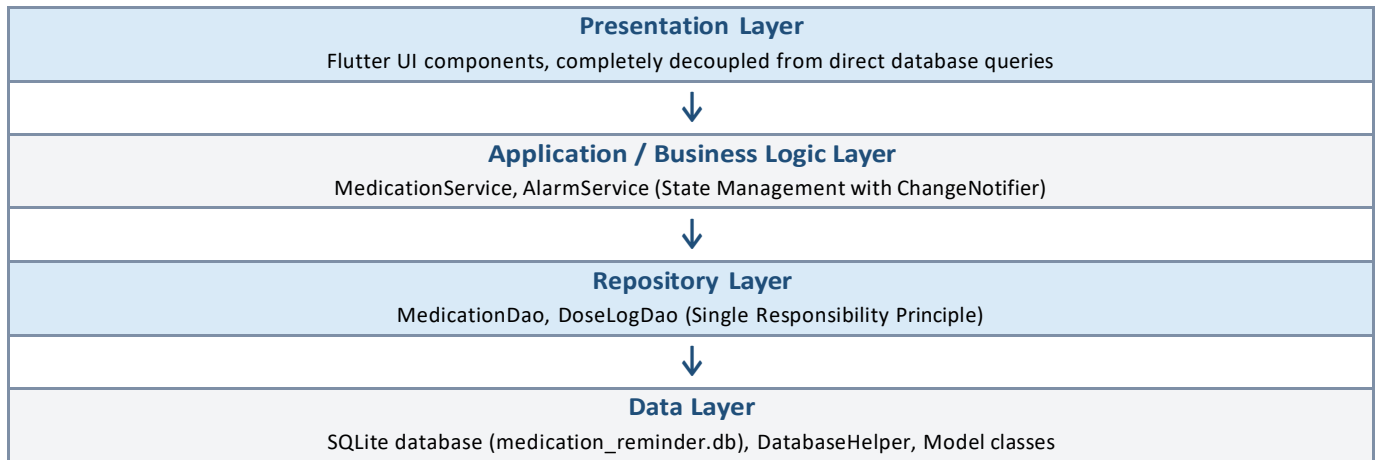


Figure 1. Suggested layered architecture flow. Students may replace this with their own architecture diagram.

[Explain how your layers communicate and why this structure improves maintainability, testability, and separation of concerns.]

Layered architecture flow of Medician Reminder.

The application is strictly designed following the layered architecture principles, ensuring a clean and maintainable separation between the Presentation (UI), Business/Domain, and Data layers. The UI components are completely decoupled from direct database queries. Instead, the UI communicates exclusively with the Business layer (specifically MedicationService and AlarmService), which in turn safely coordinates all data transactions with the underlying Data layer.

5.2 Folder and Package Structure

Paste your actual project structure or a simplified version. Do not include unnecessary generated folders.

```
lib/[cite: 3]
├─ business/[cite: 3]
│   ├── alarm_service.dart[cite: 3]
│   └─ medication_service.dart[cite: 3]
├─ dao/[cite: 3]
│   ├── dose_log_dao.dart[cite: 3]
│   └─ medication_dao.dart[cite: 3]
├─ data/[cite: 3]
│   └─ database_helper.dart[cite: 3]
├─ models/[cite: 3]
│   ├── dose_log.dart[cite: 3]
│   └─ medication.dart[cite: 3]
├─ router/[cite: 3]
│   └─ app_router.dart[cite: 3]
├─ ui/[cite: 3]
│   ├── add_medication_screen.dart[cite: 3]
│   ├── alarm_screen.dart[cite: 3]
│   ├── dashboard_screen.dart[cite: 3]
│   ├── history_screen.dart[cite: 3]
│   └─ settings_screen.dart[cite: 3]
└─ main.dart[cite: 3]
```

5.3 Responsibilities of Each Layer

Layer / component	Responsibility in this project	Example class or file
Presentation Layer	/Displays data and captures user input. Listens to the in-memory lists (<code>_medications</code> and <code>_logs</code>) and rebuilds synchronously without lagging using <code>notifyListeners()</code> .	<code>dashboard_screen.dart[cite: 3]</code> , <code>add_medication_screen.dart[cite: 3]</code>
State Management / Logic	Maintains the reactive in-memory state of the app using the <code>ChangeNotifier</code> pattern. Safely coordinates all data transactions with the underlying Data layer in the background.	<code>alarm_service.dart[cite: 3]</code> , <code>medication_service.dart[cite: 3]</code>
Repository	Abstracts the DAOs from the UI, ensuring the presentation layer does not handle raw data logic. Serves as the central Domain service.	<code>medication_service.dart[cite: 3]</code>
DAO	Encapsulates direct SQL execution and low-level data access logic to strictly uphold the Single Responsibility Principle.	<code>medication_dao.dart[cite: 3]</code> , <code>dose_log_dao.dart[cite: 3]</code>
SQLite Database	Manages local data persistence through a robust singleton initialized relational database (<code>medication_reminder.db</code>).	<code>database_helper.dart[cite: 3]</code>
Model / Entity	Represents the core domain model objects. Equipped with <code>toMap</code> and <code>fromMap</code> factory methods to handle seamless JSON/Map serialization for SQLite.	<code>medication.dart[cite: 3]</code> , <code>dose_log.dart[cite: 3]</code>

6. Data Layer: SQLite, DAO, and Repository

6.1 SQLite Database Design

Local data persistence is managed using the `sqlite` package, initialized through a robust singleton `DatabaseHelper` class. The relational database (`medication_reminder.db`) is structured around two primary tables that represent the core domain model objects.

Table name	Column	Data type	Constraint / purpose
medications	<i>id</i>	Primary Key	Acts as the primary identifier for each medication.
medications	name, dosage, frequency, times	Text/Data	Stores the configuration data of each drug.
medications	stockCount, lowStockThreshold	Numeric/Data	Inventory data used to proactively prevent the user from running out of medication.
Medications	isActive	Boolean/Data	Indicates if the medication schedule is currently active.
dose_logs	<i>id</i>	Identifier	Unique identifier for the log entry
dose_logs	medicationId	Relational link	Acts as a relational link to the medications table.
dose_logs	loggedDateTime	ISO8601 string	Stored as an ISO8601 string for accurate chronological querying.
dose_logs	action status	String	Records the user's action status ('taken' or 'skipped').

Local data persistence is managed using the `sqlite` package, initialized through a robust singleton `DatabaseHelper` class. The relational database (`medication_reminder.db`) is structured around two primary tables that represent the core domain model objects.

6.2 DAO Design

Direct SQL execution and low-level data access logic are encapsulated within dedicated DAO classes to strictly uphold the Single Responsibility Principle

DAO method	Purpose	SQL operation	Called by
<code>decrementStock()</code>	Mathematically ensures inventory does not drop below zero.	UPDATE	MedicationService
<code>getLogsForDate()</code> / <code>getAllLogs()</code>	Executes SQL queries ordered by the logged <code>DateTime</code> and utilizes <code>LIKE</code> clauses on date prefixes to fetch daily logs efficiently.	SELECT	MedicationService
<code>deleteLogsForMedication()</code>	A helper to perform cascading deletions when a medication profile is removed.	DELETE	MedicationService
<code>deleteAllLogs()</code>	Truncates the table to wipe the log history.	DELETE	MedicationService
<code>insertLog()</code> / <code>insertMedication()</code>	Handles log insertions and registers new medication profiles into the database.	INSERT	MedicationService

6.3 Repository Design

In the Medication Reminder application, the `MedicationService` class serves as the central Repository and Domain service. It abstracts the DAOs from the UI, ensuring that the presentation layer never handles raw data logic or direct SQLite queries. To solve database latency issues, the `MedicationService` is fully utilized as a reactive repository; it maintains the `_medications` and `_logs` lists in memory, which the UI simply listens to.

The service exposes intention-revealing methods such as `markAsTaken`, `addMedication`, and `clearHistory`. These methods encapsulate the business rules and coordinate with multiple DAOs. For instance, a single call to `markAsTaken` safely coordinates a structured business transaction: it inserts a log record via `DoseLogDao`, decrements the inventory stock via `MedicationDao`, and reloads the lists to propagate changes, completely hiding these sequential SQLite operations from the UI.

6.4 Data Flow Example

Describe one complete data flow from user interaction to SQLite and back to the UI.

Step	Layer/component	What happens
1	UI screen	The user marks a medication dose as "taken".
2	State management / controller	The <code>markAsTaken</code> function is triggered inside the <code>MedicationService</code> to execute a structured business transaction.
3	Repository	The service finally reloads the in-memory lists to propagate the changes.
4	DAO	The service first inserts a log record via <code>DoseLogDao</code> .
5	SQLite database	The service subsequently decrements the inventory stock via <code>MedicationDao</code> .
6	UI screen	The service gracefully updates the UI using <code>notifyListeners()</code> once the data is ready.

7. Implementation Details

7.1 Flutter Widgets and Screens

Key screens include the Dashboard (landing page), Add/Edit Medication Form, History and Logs, Medication Detail, and Settings. The UI utilizes the `colorHex` property stored in the database to color-coordinate the UI elements dynamically on screens like the Medication Detail screen. UI components are decoupled from direct database queries to avoid skipped frames and ensure smooth rendering.

7.2 Navigation and Routing

Navigation relies on a centralized routing mechanism defined in `app_router.dart`. When navigating to specific detail views, such as the Medication Detail screen, the application passes the `Medication` model object directly as a route argument to display in-depth details like instructions, `pillType`, and the current `stockCount`.

7.3 State Management

The application implements state management using the `ChangeNotifier` pattern, an industry-standard approach in Flutter. Both `MedicationService` and `AlarmService` extend `ChangeNotifier` to maintain the reactive in-memory state of `_medications`, `_logs`, and active alarms. Upon any data mutation, the `notifyListeners()` method is invoked, triggering a targeted UI update. Global instances of these services are

initialized in the main entry point, ensuring a consistent, synchronous, and predictable data flow across all `StatefulWidget` trees.

7.4 Validation, Error Handling, and User Feedback

Situation	How the app handles it	User feedback
Asynchronous database bottlenecks	The UI is isolated from data fetches. <code>MedicationService</code> acts as a reactive repository and handles complex SQLite INSERT and UPDATE operations in the background.	The UI listens to in-memory lists and rebuilds synchronously without lagging or skipping frames once data is ready.
Alarm concurrency issues	Implements a logical <code>Set<String>_triggeredAlarms</code> mechanism within <code>AlarmService</code> . It generates a unique composite key (<code>medicationId_time_date</code>) to check before triggering audio.	Ensures each medication alarm fires strictly once per day, preventing overlapping audio playback and poor user experience.
Clearing data history	The Settings screen triggers the <code>clearHistory()</code> function inside <code>MedicationService</code> , which truncates the log table.	Immediately updates the global state via <code>notifyListeners()</code> to reflect an empty history log.
Handling scheduled alarms	A background <code>Timer.periodic</code> continuously checks scheduled times.	Plays an audio alert (<code>alarm.mp3</code>) and provides the flexibility to snooze for 5 minutes or dismiss completely.

7.5 Important Code Snippets

Include only short, relevant snippets. Do not paste the entire project. Explain each snippet briefly.

Snippet title	File	Purpose / explanation
Database initialization	<code>database_helper.dart</code>	Initializes local data persistence using the <code>sqlite</code> package through a robust singleton <code>DatabaseHelper</code> class.
DAO query example	<code>medication_dao.dart</code> , <code>dose_log_dao.dart</code>	Encapsulates direct SQL execution, such as the <code>decrementStock</code> function and LIKE clauses on date prefixes to fetch daily logs efficiently.
Repository method	<code>medication_service.dart</code> , <code>alarm_service.dart</code>	Shows how the service abstracts DAOs and coordinates structured business transactions like <code>markAsTaken</code> by inserting logs and decrementing stock.
UI-state connection	<code>dashboard_screen.dart</code> , <code>alarm_service.dart</code>	Shows how the UI layer listens to the in-memory lists and rebuilds synchronously without lagging when <code>notifyListeners()</code> is invoked.

8. Testing and Quality Assurance

Explain how you tested the application and verified that important features work correctly.

Test ID	Feature tested	Test steps	Expected result	Actual result
T-01	Add medication protocol	Open Add/Edit Medication Form, enter details (name, dosage, frequency), and submit.	Data is serialized and inserted into the SQLite <code>medications</code> table via <code>MedicationDao</code> .	Pass

Test ID	Feature tested	Test steps	Expected result	Actual result
T-02	Mark dose as taken	Tap on a scheduled medication to mark it as "taken".	A log record is inserted, inventory stock is decremented by one unit, and the UI updates.	Pass
T-03	Smart Alarm trigger	Wait for a scheduled medication time to arrive.	An audio alert (alarm.mp3) is triggered asynchronously exactly once, preventing duplicate alarms.	Pass
T-04	Clear log history	Navigate to the Settings screen and tap the clear data option.	deleteAllLogs() is called to truncate the table and the global state updates immediately.	Pass
T-05	Trigger database fetch operations.	Trigger database fetch operations.	The UI listens to in-memory lists and rebuilds synchronously without lagging or skipping frames.	Pass

[Briefly discuss manual testing, emulator/device testing, edge cases, and any automated tests if implemented.]

Testing was performed manually to ensure core features work correctly. Each flow above was exercised, including specific edge cases such as alarm concurrency, where a `Set<String> _triggeredAlarms` mechanism was successfully tested to ensure the same alarm does not trigger multiple times within the same minute. Furthermore, asynchronous database bottlenecks were tested to confirm that the UI handles complex SQLite queries (like calculating compliance rates or fetching daily logs) in the background without causing lag.

9. Challenges, Design Decisions, and Solutions

9.1 Technical Challenges Encountered

Be specific. A good report explains not only what was built, but also what difficulties were encountered and how they were solved.

Challenge	Cause / context	Solution applied	Result / lesson learned
SQLite database initialization issue	Managing concurrent connections or repeated initializations to the <code>medication_reminder.db</code> relational database could cause database locks or data corruption.	Local data persistence is managed using the <code>sqflite</code> package, initialized strictly through a robust singleton <code>DatabaseHelper</code> class.	Ensures a single, stable, and predictable database connection across the entire application lifecycle without initialization conflicts.
State update or UI refresh problem	Initially, the UI layer suffered from lag and skipped frames because it was attempting to wait for complex SQLite queries (like fetching daily logs or calculating compliance rates) to finish rendering.	The <code>MedicationService</code> was fully utilized as a reactive repository. The Service handles asynchronous database INSERT and UPDATE operations in the background and gracefully updates the UI using <code>notifyListeners()</code> .	The UI now simply listens to the <code>_medications</code> and <code>_logs</code> lists in memory, updating synchronously without lagging once the data is ready.

Challenge	Cause / context	Solution applied	Result / lesson learned
<i>Navigation or argument passing issue</i>	Displaying in-depth details (like instructions, pillType, and current stockCount) requires passing complex data reliably between screens when a user taps on a specific medication.	Navigation routes the user to the Medication Detail screen, passing the complete Medication model object directly as an argument.	Ensures the detail screen has immediate access to all necessary data, allowing it to visually utilize the colorHex property stored in the database to color-coordinate UI elements dynamically.

9.2 Design Decisions

Explain important decisions and justify why you made them. Mention alternatives if relevant.

Decision	Alternatives considered	Reason for final choice
<i>[Use SQLite for local storage]</i>	<i>SharedPreferences or Cloud Database(Firebase)</i>	Local data persistence via sqflite with a robust singleton DatabaseHelper allows structured relational tables (medications and dose_logs) and ensures the app works completely offline for reliable medication tracking.
<i>[Use Repository pattern]</i>	Direct DAO or SQLite calls from the UI	The MedicationService class serves as the central Repository and Domain service. It abstracts the DAOs (MedicationDao, DoseLogDao) from the UI, ensuring the presentation layer never handles raw data logic.
<i>[Use chosen state management method]</i>	setState only, BLoC, or Riverpod	Implemented the ChangeNotifier pattern, an industry-standard approach in Flutter. Both MedicationService and AlarmService extend ChangeNotifier to maintain the reactive in-memory state of _medications, _logs, and active alarms. It updates the UI synchronously via notifyListeners() without database lag.
<i>[Use specific navigation approach]</i>	Basic routing relying on ID lookups or global state only	When navigating to the Medication Detail screen, the routing passes the complete Medication model object directly as an argument. This ensures the detail screen has immediate, synchronous access to complex data like pillType, stockCount, and colorHex without requiring additional asynchronous database fetches.

9.3 Known Limitations

While the Medician Reminder application successfully manages daily medication routines, there are a few known limitations and edge cases.

Primarily, the application relies on a `Timer.periodic` background loop for the alarm system. This approach is vulnerable to Android's strict battery optimization protocols (Doze mode), meaning alarms might be delayed or suppressed if the operating system forcibly kills background tasks or puts the device into deep sleep. Secondly, the scheduling module currently supports daily, alternate days, and specific days, but edge cases involving highly complex regimens (e.g., fractional dosages like "half a pill" or dynamically scaling dosages based on test results) are not fully handled.

In terms of performance and architecture, the application operates entirely offline using local SQLite storage. Because there is no cloud synchronization or user authentication, if the device is lost or the app data is cleared, the user's entire schedule and adherence history cannot be recovered. Furthermore, while the `ChangeNotifier` state management is reactive and prevents database lag, keeping all historical `dose_logs` in memory could eventually hit performance limitations if the user accumulates years of log data without using the "Clear History" function. These were intentional scope decisions to prioritize a standalone, privacy-focused local demo application, with native background services and REST API cloud backups planned as future improvements.

10. Conclusion and Future Work

10.1 Conclusion

The Medication Reminder project successfully functions as a comprehensive, offline medication management Android application. It achieves this by strictly adhering to a clean three-layer architecture (Presentation, Business/Domain, Data), utilizing a real SQLite database (`medication_reminder.db`) for robust local persistence, implementing DAO and Repository design patterns, and applying reactive state management via the `ChangeNotifier` pattern.

The project fully met its primary goal of delivering a reliable digital mechanism to minimize missed doses, improve treatment adherence, and proactively track medication inventory through critical stock thresholds. By avoiding reliance on a backend server, the application ensures that users can manage their highly personal health data securely and access their dynamic schedules completely offline, successfully addressing the global health problems of polypharmacy and missed medications.

Furthermore, the development process reinforced critical practical lessons in mobile software engineering. Overcoming specific challenges—such as preventing asynchronous SQLite database bottlenecks from lagging the UI and solving alarm concurrency issues to stop overlapping audio triggers—provided deep, hands-on insights. It proved the absolute necessity of decoupling the UI from raw data fetches, utilizing the `MedicationService` as a reactive in-memory repository, and safely managing background `Timer.periodic` loops. Ultimately, the project not only resulted in a functional, user-centric health utility app but also served as an excellent practical application of advanced Flutter development principles, proper state synchronization, and robust local database management.

10.2 Future Improvements

Improvement idea	Why it is useful	Possible implementation approach
<i>Cloud synchronization</i>	Allows data backup, multi-device usage, and prevents total data loss if the device is lost or the app is uninstalled.	Integrate a REST API/backend service or Firebase Cloud Firestore to securely sync local SQLite data to the cloud.
<i>Authentication</i>	Protects sensitive health data, personalizes medication schedules, and allows for multiple user profiles (e.g., managing a dependent's medicine).	Implement Firebase Auth or a custom secure login system with OAuth2 integration.
<i>Advanced search/filtering</i>	Helps users quickly locate specific medications in a large history, filter logs by date range, or sort inventory by critical stock levels.	Expand DAO methods with advanced WHERE clauses and implement a reactive search bar in the UI with a debouncer.
<i>UI/UX improvements</i>	Improves the overall user experience and accessibility, which is crucial for elderly patients who might have visual impairments.	Implement Material 3 motion animations, larger typography options, high-contrast themes, and richer empty states.

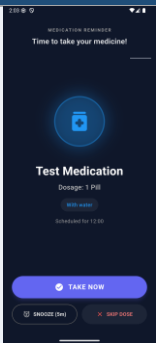
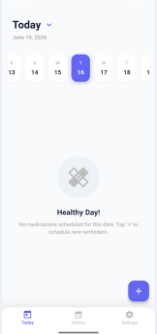
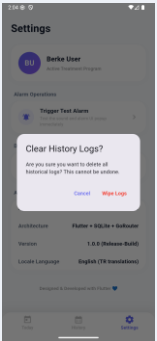
11. References

List all sources used, including official Flutter documentation, packages, tutorials, images, icons, external code snippets, or AI tools if your instructor requires disclosure.

Source	Type	How it was used
Flutter documentation (flutter.dev)	<i>Official documentation</i>	Used for widgets, navigation, and core UI implementation.
sqlite & provider (pub.dev)	<i>Package documentation</i>	Used for local SQLite access and reactive state management (ChangeNotifier).
Dart documentation (dart.dev)	Official documentation	Used for asynchronous programming (Timer.periodic) and core language features.
Audio asset (alarm.mp3)	Local media asset	Used as the audio alert for the smart alarm mechanism

12. Appendices

Appendix A: Additional Screenshots

Screen name	Screenshot placeholder	Purpose / main function	Navigation / user actions
Alarm Dialog		[Displays the active medication reminder when the background timer triggers.	The user is provided with the flexibility to snooze the alarm for 5 minutes or dismiss it completely.
Screen name	Screenshot placeholder	Purpose / main function	Navigation / user actions
Empty Dashboard State		[Insert Empty State Screenshot]	Displays a friendly visual prompt when the user has no scheduled medications for the current day.
Clear Data Confirmation		[Insert Confirmation Screenshot]	Prevents accidental deletion of the user's chronological dose logs from the database.

Appendix B: Repository and Submission Links

Item	Link / note
Source code repository	[Paste URL]
Demo video	[Paste URL if available]
APK / release file	[Paste URL if requested]
Other files	[Paste URL or describe attachment]

Appendix C: Final Self-Check Before Submission

- [+] The report clearly explains the project purpose, problem addressed, scope, target users, and core features.
- [+] Main functional screens are shown with screenshots, captions, screen purposes, user actions, and navigation paths.
- [+] The layered architecture is explained clearly, including UI, business/domain, data, DAO, Repository, and SQLite responsibilities.
- [+] SQLite tables, model/entity objects, DAO methods, Repository methods, data flow, and state flow are explained in detail.
- [+] Technical challenges, causes, design decisions, applied solutions, testing evidence, limitations, and future improvements are discussed.
- [+] Testing evidence is included.
- [+] References and external resources are acknowledged.
- [+] Formatting, grammar, page numbering, and figure/table captions have been checked.